



# Archeos Society

Trabalho GPMS | 2ª Apresentação

Diego Cruz Camargo | Felipe Sepulveda Engel

# Agenda

01

## Visão Geral do Produto

Status da entrega e demo da versão final

02

## Controle de Versões

Repositório, estratégia de ramificação e fluxo de trabalho

03

## Controle de Modificações

Gestão de tarefas no ClickUp

04

## Dificuldades e Decisões

Saída de integrante e migração tecnológica

05

## Monitoramento e Controle

Burndown, Gantt Planejado vs Real e Análise de Valor Agregado

06

## Resultados e Lições Aprendidas

Indicadores finais do projeto

# Status da Entrega



100%

Escopo Entregue



40h / 40h

Horas Realizadas vs Planejadas



0

Dias de Atraso

## Resumo Executivo

Apesar da saída de um integrante logo no início e da consequente migração de stack tecnológica, o grupo entregou **100% do escopo planejado, dentro do prazo e dentro do orçamento de mão de obra estimado**. O replanejamento ágil das tarefas, somado à decisão técnica de unificar frontend e backend em Next.js, foi determinante para o sucesso da entrega.



## GitHub

Hospedagem do código-fonte, artefatos do projeto e histórico de modificações

[github.com/felipeSepulvida/Archeos-Society-GPMS](https://github.com/felipeSepulvida/Archeos-Society-GPMS)



### Código-fonte

Aplicação Next.js + TypeScript com toda a lógica do jogo



### Artefatos

EAP, cronograma, matriz de riscos e documentação técnica



### Licença MIT

Código público com licença aberta conforme especificação

# Estratégia de Ramificação

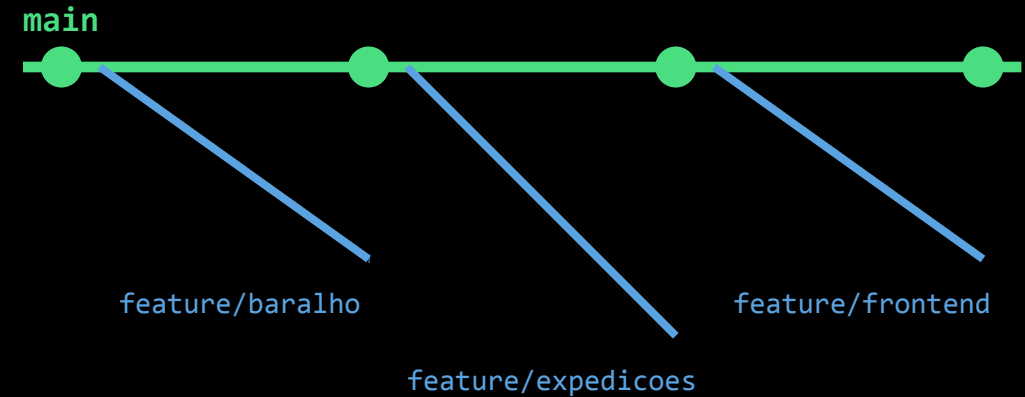
## Modelo Adotado: GitHub Flow Simplificado

### main

Branch principal, sempre estável. Reflete o estado de produção do projeto.

### feature/\*

Branches de trabalho por funcionalidade. Após conclusão, merge direto em main.



### Justificativa da escolha

Com apenas 2 desenvolvedores ativos no projeto, modelos complexos como GitFlow (com develop, release, hotfix) gerariam mais sobrecarga de processo do que valor. O fluxo simplificado main + feature/\* manteve a rastreabilidade do histórico e o isolamento entre tarefas, sem fricção desnecessária.

*Cada feature isolada em sua branch -> merge na main após conclusão*



### Limitações reconhecidas

Não foram utilizados Pull Requests com revisão obrigatória nem proteção da branch main. Em uma equipe maior, esses controles seriam essenciais para garantir qualidade do código.

# Controle de Modificações | ClickUp

## Por que ClickUp?

Optamos pelo ClickUp em vez do GitHub Issues por três razões principais:

1

### Visão consolidada

Integração nativa entre Gantt, lista de tarefas e estimativas em uma única ferramenta.

2

### Estimativas e tempo

Campos próprios para horas previstas e realizadas, sendo fundamental para o monitoramento exigido.

3

### Visualizações múltiplas

Mesmo conteúdo em modo Gantt, Board (Kanban) e Lista, conforme o contexto.



### O que registramos para cada tarefa



**Título e descrição:** Identificação clara do trabalho a ser feito



**Responsável:** Membro alocado à tarefa



**Estimativa em horas:** Esforço previsto (Planning Poker)



**Horas realizadas:** Tempo efetivamente gasto



**Status:** Aberto, em andamento, concluído



**Datas de início e fim:** Para alimentar o Gantt



**Dependências:** Pré-requisitos entre tarefas

# Análise Crítica | Práticas Não Adotadas

Em um projeto profissional, três práticas seriam essenciais e foram deixadas de fora:

## Pull Requests

Não adotado

Merges diretos em main, sem revisão de código entre pares.

### IMPACTO

Em equipes maiores, PRs garantem qualidade, conhecimento compartilhado e rastreabilidade das mudanças. Com 2 pessoas em comunicação síncrona constante, o custo do PR superou o benefício.

## GitHub Actions (CI)

Não adotado

Sem pipeline de integração contínua, testes automatizados ou deploy automático.

### IMPACTO

Em um projeto com testes automatizados, CI rodaria a suíte a cada push. Como a validação foi majoritariamente manual, a ausência teve menos impacto no curto prazo.

## Code Review formal

Não adotado

Revisões aconteceram informalmente, via conversa entre os membros.

### IMPACTO

Em projetos reais, code reviews documentados ajudam a manter padrões e disseminar conhecimento técnico.

# Dificuldade 1 | Saída de Integrante



## O que aconteceu

Logo no início do projeto, um dos três integrantes deixou a disciplina. O grupo passou de 3 para 2 pessoas, mantendo o mesmo escopo e o mesmo prazo definidos originalmente.

3

integrantes



2

integrantes

*Carga de trabalho por desenvolvedor aumentou em 50%*



## Risco previsto na matriz inicial

### Risco:

Ausência de integrante

Probabilidade: 20% - Impacto: 1,0 - Exposição: 0,2

### Contingência prevista:

*Realocação das tarefas para os outros membros e replanejamento dos prazos.*

## Resposta executada

### Replanejamento de prazos:

tarefas redistribuídas entre os 2 membros restantes.

### Decisão técnica de suporte:

migração para Next.js (próximo slide) para reduzir o esforço total.

### Resultado:

**100% do escopo entregue dentro do prazo original.**

## Decisão 2 | Migração para Next.js

Para reduzir o esforço total e compensar a perda de capacidade da equipe, decidimos consolidar frontend e backend em uma única aplicação Next.js.

### ANTES

#### React + TypeScript + Flask (Python)

*Arquitetura cliente/servidor separada com comunicação via API REST.*

- ✗ Dois projetos para manter (frontend e backend)
- ✗ Necessidade de definir e implementar contratos de API REST
- ✗ Gerenciamento de CORS, serialização, autenticação dupla
- ✗ Mais código de glue entre as duas pontas
- ✗ Maior esforço total de integração



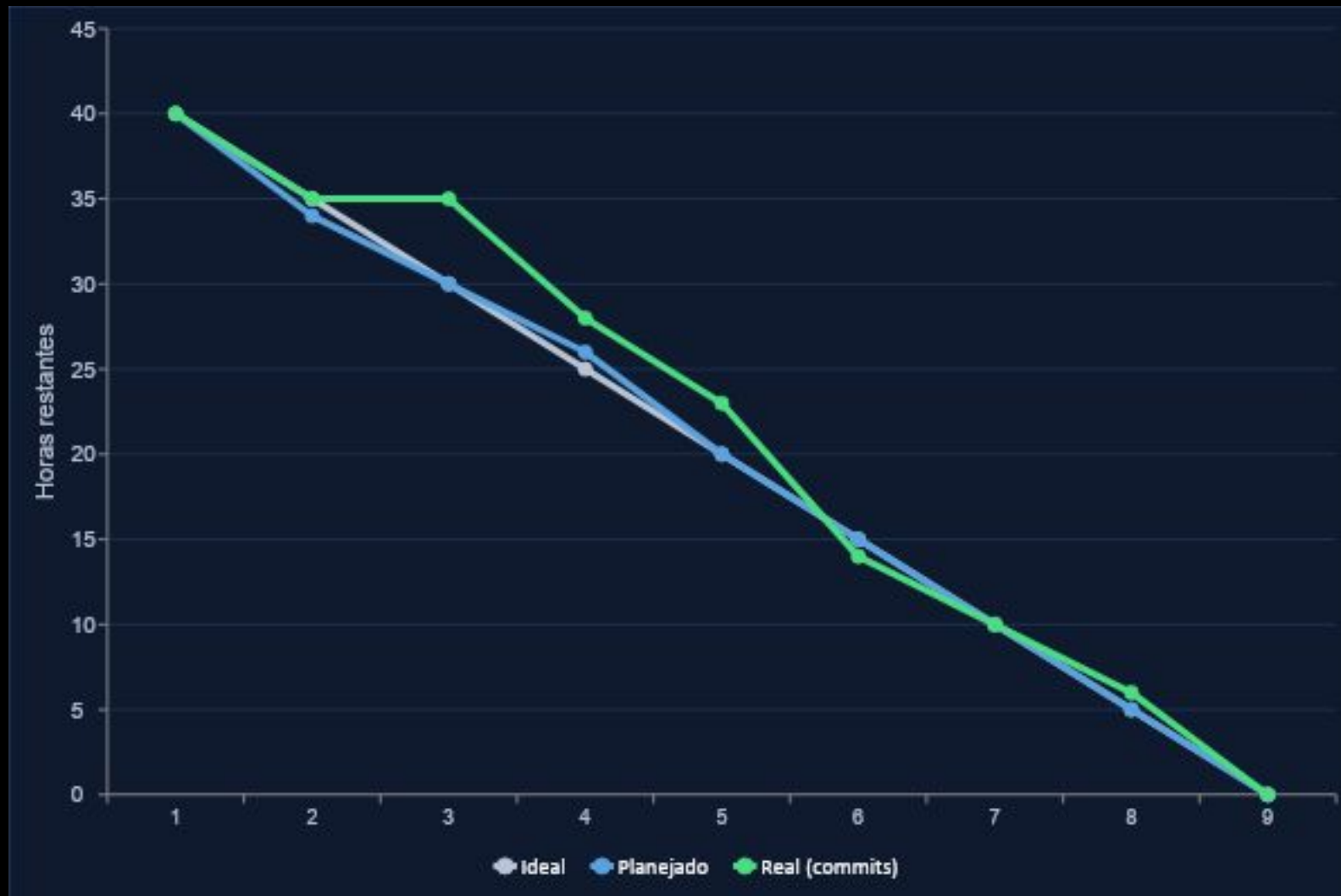
### DEPOIS

#### Next.js + TypeScript (full-stack)

*Aplicação única com Server Components e API Routes integrados.*

- ✓ Um único projeto, uma única stack (TypeScript)
- ✓ Server Actions eliminam o boilerplate de API REST
- ✓ Tipagem compartilhada entre cliente e servidor
- ✓ Menos código de glue, mais foco na lógica do jogo
- ✓ Esforço total de desenvolvimento significativamente menor

# Monitoramento | Burndown do Projeto



## Leitura

### Semana 1

Pequeno atraso entre 15-18/04 no desenvolvimento do baralho. Linha real fica acima da planejada.

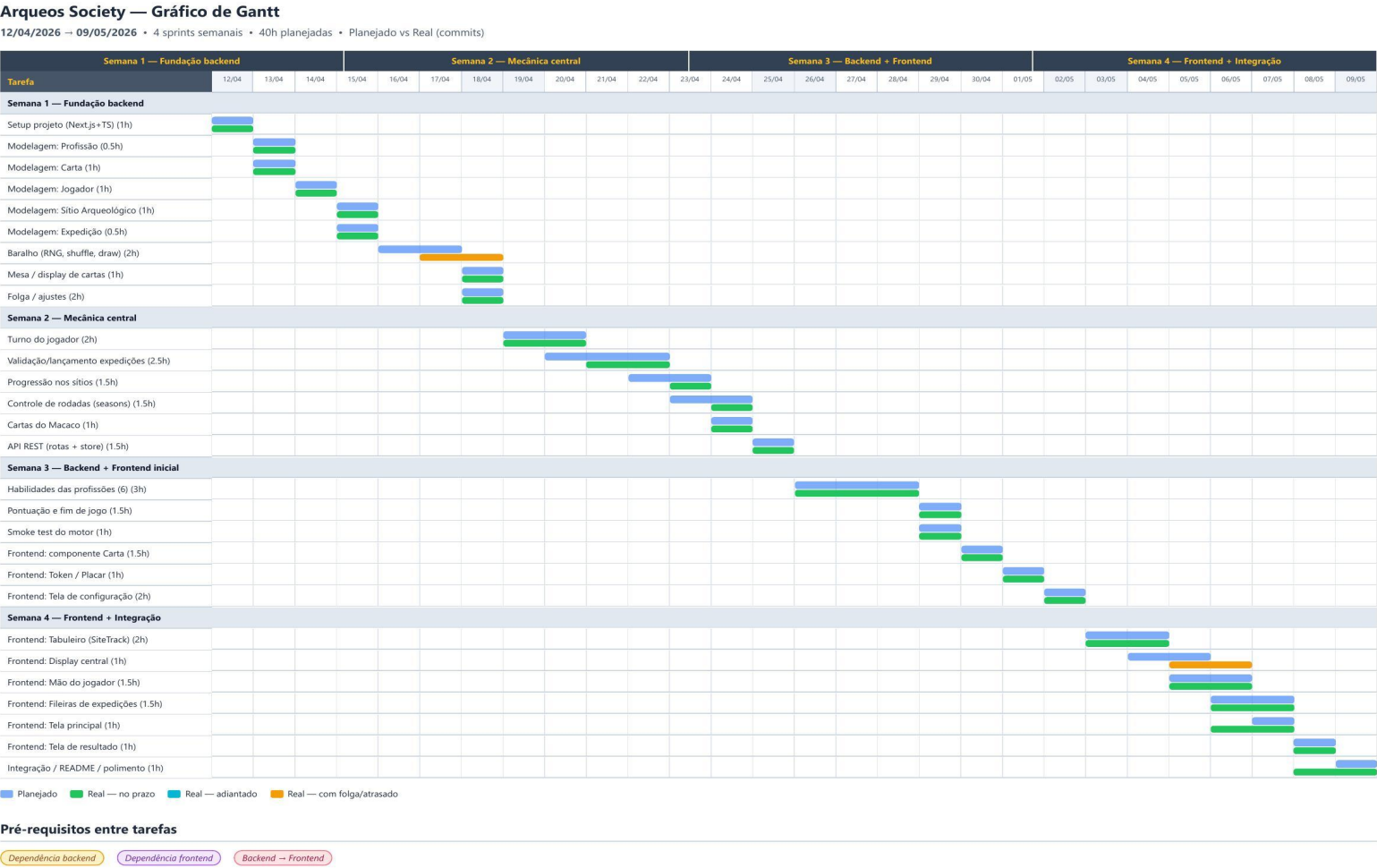
### Semana 2

Recuperação completa do atraso. Linhas se reaproximam.

### Semanas 3 e 4

Execução em linha com o planejado. Entrega final em 09/05, zerando as horas restantes no prazo.

# Monitoramento | Gantt Planejado vs Real



## Resumo



Planejado



Real - no prazo



Real - atrasado

## Pontos críticos

### Baralho (S1):

atraso pontual no shuffle/RNG, recuperado.

### Display/Mão (S4):

folga consumida no fim do projeto, entregue dentro do prazo.



Entrega: 09/05 conforme cronograma original

# Análise de Valor Agregado (EVA)

**Premissas:** BAC (Budget at Completion) = R\$ 2.000,00 | 40h × R\$ 50/h (mão de obra) | • 4 sprints × 10h. Cada sprint representa 25% do trabalho planejado.

| Sprint | Período       | PV (R\$) | EV (R\$) | AC (R\$) | SV (R\$) | CV (R\$) | SPI  | CPI  |
|--------|---------------|----------|----------|----------|----------|----------|------|------|
| S1     | 12/04 - 18/04 | 500,00   | 460,00   | 525,00   | -40,00   | -65,00   | 0,92 | 0,88 |
| S2     | 19/04 - 25/04 | 1.000,00 | 1.000,00 | 1.025,00 | 0,00     | -25,00   | 1,00 | 0,98 |
| S3     | 26/04 - 02/05 | 1.500,00 | 1.500,00 | 1.500,00 | 0,00     | 0,00     | 1,00 | 1,00 |
| S4     | 03/05 - 09/05 | 2.000,00 | 2.000,00 | 2.000,00 | 0,00     | 0,00     | 1,00 | 1,00 |

## Fórmulas e leitura

**PV (Planned Value)** Valor planejado para o trabalho previsto até a data

**EV (Earned Value)** Valor do trabalho efetivamente concluído

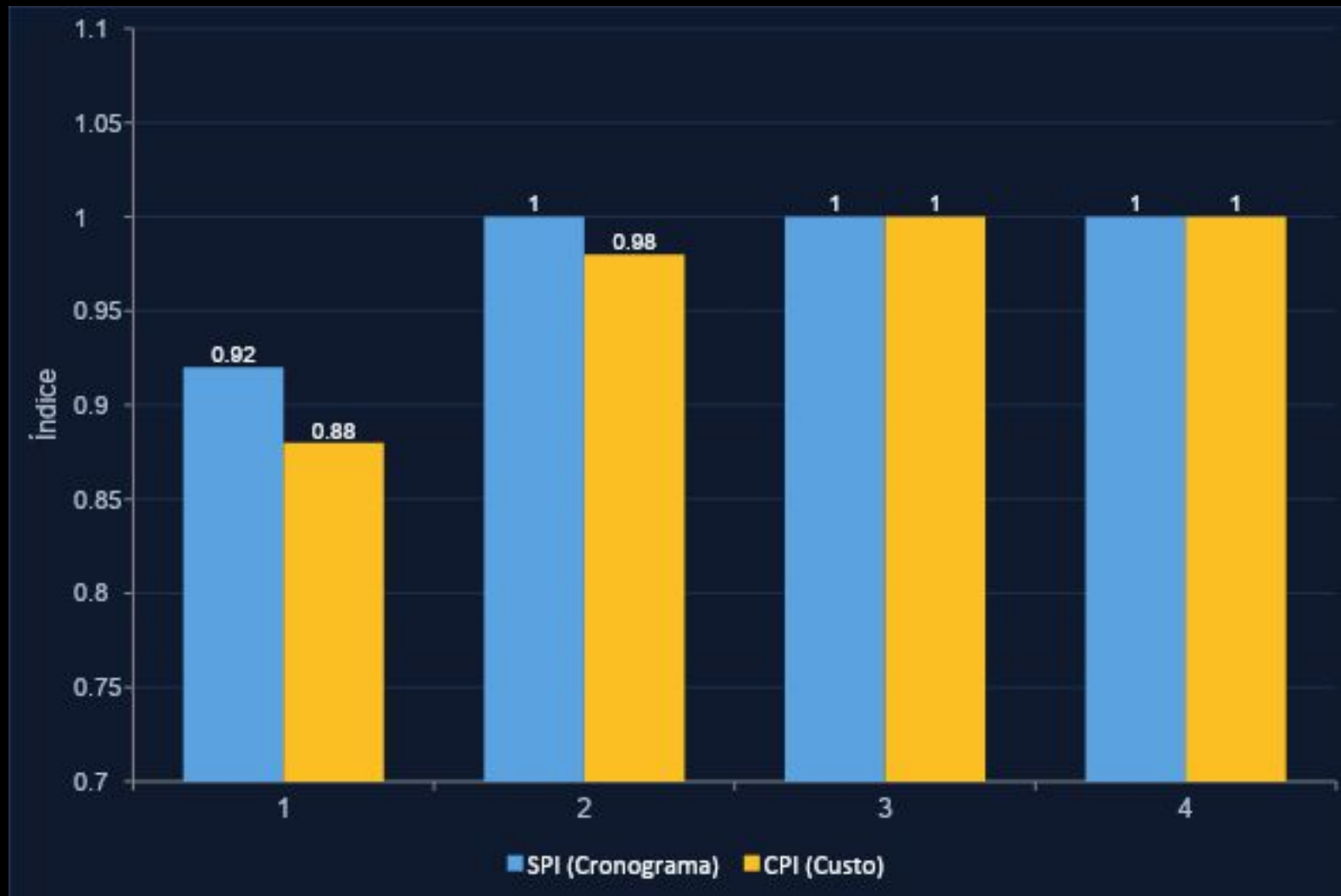
**AC (Actual Cost)** Custo real incorrido até a data

**SV = EV – PV** Variação de cronograma. > 0 adiantado, < 0 atrasado

**CV = EV – AC** Variação de custo. > 0 abaixo, < 0 acima do orçamento

**SPI = EV / PV** Índice de desempenho de cronograma. 1,0 = no prazo

# EVA | Índices de Desempenho por Sprint



## Indicadores Finais

CPI final

**1,00**

*Eficiência de custo ideal*

SPI final

**1,00**

*Cronograma cumprido*

EAC

**R\$ 2.000**

*Estimativa ao término*

VAC

**R\$ 0**

*Variância no término*

# Resultados Finais

*Mesmo com a saída de um integrante e a migração de stack, o projeto foi entregue dentro do escopo, do prazo e do orçamento.*



100%

Escopo entregue



40h

Horas planejadas e realizadas



1,00

SPI



1,00

CPI



0

Riscos materializados não tratados



0

Dias de atraso na entrega

# Lições Aprendidas

01

**A análise de riscos prévia teve valor concreto**



O risco "Ausência de integrante" foi identificado no planejamento com contingência explícita. Quando ele se materializou, a equipe já sabia o que fazer evitando uma paralisia, apenas execução do plano.

02

**Decisões técnicas devem responder a restrições do projeto**



A migração para Next.js não foi motivada por preferência tecnológica, mas pela necessidade de reduzir esforço total após a redução da equipe. Boas decisões técnicas servem ao gerenciamento, não competem com ele.

03

**Processo deve ser proporcional ao tamanho do time**



Optar por GitHub Flow simplificado em vez de GitFlow, e abrir mão de PRs e CI, foram escolhas conscientes para um time de 2 pessoas. Em um time maior ou em produção, essas práticas seriam obrigatórias.

04

**Monitoramento contínuo evita surpresas no final**



O acompanhamento semanal via Burndown e o registro detalhado de horas no ClickUp permitiram identificar o atraso da Sprint 1 ainda no início e replanejar a Sprint 2 para recuperá-lo.



## Demo ao vivo

Versão final da aplicação Archeos Society

[github.com/felipeSepulvida/Archeos-Society-GPMS](https://github.com/felipeSepulvida/Archeos-Society-GPMS)



# Obrigado!

Perguntas e discussão

v

Archeos Society | Trabalho GPMS | 2026.1

Diego Cruz Camargo | Felipe Sepulvida Engel